

Container Management with Podman

Podman

A container management tool that lets you run, build, and manage containers and images with a command-line interface.

- Remove what you need
- Return when done
- Trade parts

What Podman Can Do

- Container management
- Image management
- Pod creation
- Storage (volumes and bind mounts)
- Networking

Podman vs. Docker

Docker

- Requires root access
- Depends on daemon
- Not for multi-user environments

Podman

- Doesn't require root access
- Doesn't rely on daemon
- Supports Docker commands

Setting Up the Lab Environment

This document provides the step-by-step instructions needed to prepare the lab environment for hands-on exercises in this course.

1. System requirements for RHEL 10

CPU

- X86_64 (Intel/AMD 64-bit) or ARM64 (aarch64)
- Virtualization extensions (Intel VT-x/AMD-V) recommended if running inside a VM

Memory

- Minimum: 2GB RAM
- Recommended: 4GB + RAM (especially if you 'all run multiple containers or podman)

Storage

- Minimum root filesystem: 10 GB free space
- Recommended: 20 GB +

Networking

- Internet access (for package installation and pulling container images)

Users

- A normal user with sudo privileges

Where to get the RHEL 10 image

You can download RHEL 10 legally and at no cost through Red Hat's Developer Program at developers.redhat.com.

- Create a free account at developers.redhat.com
- After logging in, you can download: RHEL 10.0 Binary DVD ISO (~10 GB) for installation in a VM/physical server
- The Developer Subscription entitles you to run RHEL on up to 16 physical or virtual systems for free (personal use, labs, dev/test)

Direct RHEL 10 download links (requires login)

at <https://developers.redhat.com/products/rhel/download>

Here are the steps to set up your lab:

- Register your system running the `rhc connect` command
- Run the `sudo dnf install container-tools` command to install podman

Running Podman Containers

Interactive

```
#podman run -it --name my-ubuntu ubuntu:22.04 bash
```

Detached

```
#podman run -d --name my-nginx -p 8080:80 nginx:latest
```

Pass Variables

```
#podman run -it --name my-env-app -e MYVAR=test alpine
```

Auto Remove

```
#podman run --rm -it alpine sh
```

Run container with a specific command.

```
#podman login registry.redhat.io - To login red hat registry
```

```
#podman pull registry.redhat.io/rhel8/httpd-24 - To pull image
```

#podman images - To see the images

#podman run --name test --rm registry.redhat.io/rhel8/httpd-24:latest httpd -v

(This command is used to run and exit from container with output)

Run a container interactively.

#podman run --name interactive -it registry.redhat.io/rhel8/httpd-24:latest /bin/bash

(This command is used to run the container and go inside the container)

#podman ps - To check only running container status

Run a container in detached mode.

#podman run -d --name detached registry.redhat.io/rhel8/httpd-24:latest

Managing Containers with Podman

- Listing
- Starting
- Stopping
- Inspecting
- Logging
- Removing containers

Podman Container Management Commands

#podman ps - List containers running containers only

#podman start my-nginx - Start container start the existing container only.

#podman stop my-nginx - Stop an existing container.

#podman inspect my-nginx - inspect Containers

#podman logs my-nginx - Show logs

#podman rm my-nginx - To remove container

#podman ps - To see running container status

#podman stop detached - To stop the container

#podman ps - To see running container status

#podman start detached - To start the container

#podman restart detached - To restart the container

#podman ps - To see running container status

Inspect the container

#podman inspect detached - To inspect the container

View container logs.

#podman logs detached - To see the container logs

#podman rm -f detached - To remove the running the container forcefully

#podman ps - To see the running container status.

Inspecting and Troubleshooting Containers.

Troubleshoot container problems with Podman commands and logs.

Podman troubleshooting toolset - inspect, filter, podman exec, logs, and podman stats and exit codes

#podman images - To see the images

#podman run --name failed-mariadb -d registry.redhat.io/rhel9/mariadb-105:latest

(To run the container)

#podman ps

#podman ps -a

#podman inspect -f '{{.State.ExitCode}}' failed-mariadb - To inspect then output 1

#podman logs failed-mariadb - To check the logs

#podman run -d --name working-mariadb -e MYSQL_ROOT_PASSWORD=redhat -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=redhat registry.redhat.io/rhel9/mariadb-105:latest

(To run mariadb container successfully)

Container Images and Registries

1. How container images work
2. How registries store and deliver them

Container Image: A read-only template used to create containers.

Container image

- Application
- Required libraries and dependencies
- Configuration files

What Images Are Made Of

- Base system
- App files
- Configuration

Note: Podman reuses layers to save space and speed up builds.

How Container Images Are Built

Base layer: UBI
App layer: nginx
Config layer: Custom config

Container Registries: Place to store and share container images

Public registries:

Docker Hub: docker.io
Red Hat: registry.access.redhat.com

Note: Private registry used by organizations for secure storage

Managing Container Images

Managing images means knowing what you run and keeping things clean.

Managing Images in Podman

- Access registry via authentication
- `podman login`
- Images stored under **`/var/lib/container/storage`**

`#podman tag/untag` - Rename images; prepare them for publish to a registry

`#podman rmi` - Removes unused or outdated images

`#podman prune` - Cleans up untagged images

How to inspect an image from a registry before pulling it?

Skopeo: Lets you interact with container registries and image metadata.

Skopeo inspect

Skopeo copy

#podman search httpd - To search httpd image

#skopeo inspect docker://registry.redhat.io/rhel8/httpd-24

(To inspect the httpd image without running)

#podman pull registry.redhat.io/rhel8/httpd-24 - To pull the image

#podman images

#podman inspect registry.redhat.io/rhel8/httpd-24 - To inspect

#podman tag registry.redhat.io/rhel8/httpd-24 mynewimagetag:2.0 - To tag the image

#podman images

#podman rmi registry.redhat.io/rhel8/httpd-24 - To remove the image

Building Container images

Building an image lets you control every layer from the OS to its behaviour.

To create the image, you need a **ContainerFile**.

ContainerFile

- Simple text file
- Defines how to build image

Containerfile instructions

- FROM
- COPY
- ADD
- RUN
- CMD
- ENTRYPOINT
- EXPOSE
- ENV
- WORKDIR
- USER
- LABEL

Note: - Build context is the folder Podman sends to the build process

Podman Commit: - Lets you capture a container's state and save it as a new image.

Create a Containerfile:

#mkdir workdir - To create a directory

#cd workdir - To go inside this directory

#echo "Hello World" >> index.html - To create a file

#vim Dockerfile - To create a containerfile

```
FROM registry.redhat.io/ubi8/ubi:latest
ENV PORT=8080
RUN dnf install httpd -y && dnf clean all
COPY index.html /var/www/html/index.html
RUN mkdir -p /run/httpd /var/log/httpd && chown -R apache:apache /run/httpd
/var/log/httpd /var/www/html
RUN sed -ri "s/^Listen .*/Listen ${PORT}/" /etc/httpd/conf/httpd.conf
EXPOSE 8080
USER apache
CMD ["/usr/sbin/httpd", "-DFOREGROUND", "-e", "info"]
```

:wq - To save and quit

Create the container with podman build

#podman build -t mynewimage -f Dockerfile

#podman images

#Create a container based on the image

#podman run -d --name web -p 8080:8080 mynewimage

#podman ps

#curl localhost:8080

#podman exec -it web /bin/bash - To go inside the container

Challenges:

Deploy a custom container.

Challenges Tasks

1. Create a welcome.html file with the message: "Welcome to my custom container!"
2. Write a Dockerfile using UBI8 as base image with httpd installed to serve this file.
3. Build the container image
4. Run and test the container.

Solution: Deploy a custom container

#mkdir challenge - To create a directory

#cd challenge - To go inside this directory

#echo "Welcome to my custom container!" >> index.html

#vim Dockerfile - to create a containerfile

```
FROM registry.redhat.io/ubi8/ubi:latest
ENV PORT=8080
RUN dnf install httpd -y && dnf clean all
COPY index.html /var/www/html/index.html
RUN mkdir -p /run/httpd /var/log/httpd && chown -R apache:apache /run/httpd
/var/log/httpd /var/www/html
RUN sed -ri "s/^Listen .*/Listen ${PORT}/* /etc/httpd/conf/httpd.conf
EXPOSE 8080
USER apache
CMD ["/usr/sbin/httpd" ,"-DFOREGROUND" ,"-e" ,"info"]
```

:wq - To save and exit

#podman build -t challengeimage -f Dockerfile

(To create a image using this command)

#podman images

#podman run -d --name challenge -p 8080:8080 challengeimage

(To run a container using creating image)

Introduction to Pods

How Podman pods let containers share a network.

Pod: - A group of one or more containers that run together and share network resource.

Why Use Pods?

- Ideal for containers that work together
- Build modular, reusable setups
- Reflects Kubernetes-native architecture

Shared:

- Network stack
- Loopback interface (localhost)
- Some namespaces

Isolated:

- Filesystem
- Process namespace
- User IDs

Pods let containers communicate while staying secure.

Use pods for your projects

Practice using them within Podman

Creating and Managing Pods

Managing Pods

#podman pod create <pod-name> - Create a pod

#podman run --pod <pod-name> <image> - Add new container to a pod

#podman pod ps - Monitor pods

#podman pod inspect <pod-name> - inspect a pod

#podman pod rm <pod-name> - Remove a pod

Practical:

Create a pod

#podman pod create --name mypod -p 8080:8080 - To create a pod

#podman pod ps - To see status of pod

Add first container to pod

#podman run -d --name web --pod mypod registry.redhat.io/rhel8/httpd-24:latest

Add second container to pod

#podman run -d --name client --pod mypod registry.redhat.io/ubi9/ubi:latest sleep infinity

View pods and containers

#podman pod ps

#podman ps --pod

#podman exec -it client /bin/bash - To go inside the container

#exit - To exit from the container

Clean up

#podman pod rm -f mypod

Container Storage and Networking

Ephemeral Storage: - Data disappears when container is destroyed.

Persistent Storage: - Data is stored outside the container and persists after it's destroyed.

Persistent Storage in Podman

- Volumes
- Bind mounts

Volumes

- Storage managed by podman
- Provide isolated, portable persistent storage

Bind Mounts

- Map a host folder into container
- Offer more flexibility
- Expose the container to host changes.

Networking in Podman

- Bridge mode (default)
- Host mode
- None
- Custom networks
- Pods

Open ports with port mapping.

Port Mapping: - Connects the outside world to a container's service

Port Mapping in Podman

```
#podman run -p 8080:80 apache
```

8080 ----→ Host Port

80 -----→ Container Port

```
#podman port -l          -          List port mappings
```

Practical Lab:

Start a container without port mapping

```
#podman images
```

```
#podman run -d --name web registry.redhat.io/rhel8/httpd-24:latest
```

```
#podman ps
```

Find the exposed port

```
#podman inspect registry.redhat.io/rhel8/httpd-24:latest
```

Start a container with port mapping

```
#podman run -d --name web2 -p 8080:8080 registry.redhat.io/rhel8/httpd-24:latest
```

```
#podman ps
```

```
#podman port -l          -          To list port
```

Custom Container Networks

- Build
- Connect
- Control

Note: - A custom container network is like setting up your own private LAN.

Podman Commands for Network

```
#podman network ls
```

```
#podman network create mynet
```

```
#podman network inspect mynet
```

```
#podman run -d --name web --network mynet nginx
```

```
#podman network rm mynet
```

```
#podman network connect/disconnect mynet web
```

Practical Lab:

Create a network

```
#podman network create mynetwork - To create a network
```

```
#podman network ls - To list the available network
```

Start a web server on the network

```
#podman images
```

```
#podman run -d --name web --network mynetwork registry.redhat.io/rhel8/httpd-24:latest
```

```
#podman ps
```

```
#podman exec -it web /bin/bash - To go inside the container
```

```
#echo test >> /var/www/html/index.html - To create a file inside the container
```

```
#exit - To exit from the container
```

Start a client on the same network

#podman images

#podman run -d --name client --network mynetwork registry.redhat.io/ubi8/ubi:latest
sleep infinity

#podman ps

Validate your work

#podman exec -it client /bin/bash - To go inside the container

#curl web:8080 - To curl the website inside the container

#exit - to exit from the container

#podman rm -f web - To remove container forcefully.

#podman rm -f client - To remove container forcefully.

#podman network rm mynetwork - To remove mynetwork

Using Volumes and Bind Mounts for Persistent Storage

Volumes:

- Fully managed by Podman
- Isolated
- Easy to migrate

Bind mounts:

- You control the host directory
- Direct link to host files
- Tied to host

Commands for Working with Volumes

#podman volume create mydata - Create a Volume

#podman volume ls - List Volumes

#podman volume inspect mydata - Inspect a volume

#podman volume rm mydata - Remove a volume

#podman run -d -v mydata:/var/lib/mysql mysql:latest - Run a container with volume Mounted.

Commands for bind mounts

```
#podman run -d -v ./site:/var/www/html httpd:latest  
(Map a host folder to a container)
```

```
#podman run -d -v ./config:/app/config:ro myapp  
(Map with read-only permissions)
```

Practical Lab

Create and inspect a Volume

```
#podman volume create webdata
```

```
#podman volume ls
```

```
#podman volume inspect webdata
```

Run a container with volume

```
#podman run -d --name web1 -p 8080:8080 -v webdata:/var/www/html:Z  
registry.redhat.io/rhel8/httpd-24:latest
```

```
#podman ps
```

```
#podman exec -it web1 /bin/bash - To go inside the container
```

```
#echo "I am persistent" >> /var/www/html/index.html
```

```
#exit - To exit from the container
```

```
#curl 127.0.0.1:8080
```

Recreate the container with same volume

```
#podman rm -f web1
```

```
#podman run -d --name web2 -p 8080:80 -v webdata:/var/www/html/index.html:Z  
registry.redhat.io/rhel8/httpd-24:latest
```

```
#curl 127.0.0.1:8080 - In the output is (I am persistent)
```

Use bind mount:

```
#mkdir webcontent
```

```
#echo "I am bind mount" >> webcontent/index.html
```

```
#podman run -d --name web3 -p 9091:8080 -v  
/home/student/webcontent:/var/www/html:Z httpd
```

```
#curl 127.0.0.1:9091 - The accepted command output (I am a bind mount)
```

Challenge 🧑

Note: - Deploy a multi-container application

You need to deploy a small multi-container web application. The application consists of a frontend web server and backend database. Both containers must run inside the same Pod so they can communicate.

Challenge Tasks

- Create a Pod named myapp-pod and map host port 8080 to container port 8080.
- Create a Volume named DBDATA.
- Deploy the database container (rhel8/mariadb-105) inside the Pod.
- Prepare a host folder named html with a simple index.html with the content "Hello World"
- Deploy the frontend container (httpd:latest) inside the Pod using a bind mount which binds the ./html folder to /var/www/html within the container.

Solution 😊

Deploy a multi-container application

1. Create a Pod named myapp-pod and map host port 8080 to container port 8080.

```
#podman pod create --name myapp-pod -p 8080:8080
```

```
#podman pod ls
```

2. Create a Volume named DBDATA

```
#podman volume create DBDATA
```

```
#podman volume ls
```

```
#podman volume inspect DBDATA
```

3. Deploy the database container (rhel8/mariadb-105) inside the Pod.

```
#podman search mariadb-105
```

```
#podman pull registry.redhat.io/rhel8/mariadb-105 - To pull MariaDB image
```

```
#podman images
```

```
#podman run -d --pod myapp-pod --name db -v DBDATA:/var/lib/mysql:Z -e  
MYSQL_USER=braham -e MYSQL_PASSWORD=redhat -e MYSQL_DATABASE=mydb -e  
MYSQL_ROOT_PASSWORD=redhat registry.redhat.io/rhel8/mariadb-105:latest
```

```
#podman ps
```

4. Prepare a host folder named html with a simple index.html with the content "Hello World"

```
#mkdir html
```

```
#echo "Hello World" >> html/index.html
```

5. Deploy the frontend container (httpd:latest) inside Pod using a bind mount which binds the ./html folder to /var/www/html within the container.

```
#mkdir html
```

```
#echo "Hello World" >> html/index.html
```

```
#podman run -d --name web1 --pod myapp-pod -v  
/home/student/html:/var/www/html:Z registry.redhat.io/rhel8/httpd-24: latest
```

```
#podman ps
```

```
#curl 127.0.0.1:8080
```

```
#
```

